

Lab 00 — Questions

Answer without rereading the theory. One to five sentences are enough; what counts is the reasoning, not the vocabulary.

Question 1 [Comprehension]

A binary compiled for Linux (for example `nginx`) works identically on Ubuntu, Debian and Alpine, but not at all on Windows without WSL. Explain what this binary expects from its system, and why the distribution doesn't matter while the kernel does.

Question 2 [Analysis]

You run `sleep 300 &` in a terminal, then close that terminal. A little later, `ps -ef` shows that the `sleep` process still exists, but that its PPID is now `1`. What happened, and why does the system need this mechanism?

Question 3 [Diagnosis]

A colleague shows you this:

```
$ ./deploy.sh
bash: ./deploy.sh: Permission denied
$ echo $?
126
```

The file exists and he owns it. Give the exact cause, the command that confirms it, the command that fixes it — and a second way to run the script without fixing anything at all.

Question 4 [Prediction]

Predict the two lines printed by this sequence, then justify the difference:

```
MSG=hello
bash -c 'echo 1: $MSG'
export MSG
bash -c 'echo 2: $MSG'
```

Question 5 [Diagnosis]

In an operations script you find `echo $?` printing `137` right after the brutal stop of a Java service. Decompose that number, say what happened to the process, and why this precise code is famous in the container world.

Question 6 [Analysis]

Many hurried administrators systematically use `kill -9` instead of `kill`. Explain the difference in mechanism between the two, what a database-type application concretely loses in the second case, and the link with the way Docker stops a container (`docker stop`).

Question 7 [Diagnosis]

Observe:

```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 652 Mar 31 13:31 /etc/shadow
$ sudo cat /etc/shadow    # works
```

Relying on the `ls -l` line, explain precisely why the first `cat` fails and why the second succeeds. Who could read this file without `sudo`?

Question 8 [Prediction]

What does the file `result.txt` contain and what appears on screen after this command, knowing that `/unknown-date` does not exist?

```
ls /etc/hostname /unknown-date > result.txt 2> errors.txt
```

And what would `2>&1`, placed after `> result.txt`, change?

Question 9 [Analysis]

`ss -tlnp` on a server shows these two lines:

```
LISTEN 0    511      127.0.0.1:6379    0.0.0.0:*    users:(("redis-server",pid=812,fd=6))
LISTEN 0    511      0.0.0.0:8080     0.0.0.0:*    users:(("java",pid=944,fd=23))
```

What is the difference in reach between these two services? From another machine on the network, which one can you reach? Why will this detail become important when you publish container ports?

Question 10 [Comprehension]

Your user (UID 1000) runs `python3 -m http.server 80` and gets `PermissionError: [Errno 13] Permission denied`, while port 8080 works. Explain the rule at play, its historical reason for existing, and the direct consequence for rootless Podman.

Question 11 [Analysis]

`ls /proc` shows hundreds of directories, and yet `df -h` shows no disk space consumed by `/proc`; `findmnt -t proc` reveals a filesystem of type `proc`. Explain what `/proc` really is, where its "files" come from, and give an example of information you would go looking for there.

Question 12 [Diagnosis]

On a freshly installed machine, a colleague copied a tool into `~/tools/mytool`, checked with `ls` that it is indeed executable, but gets:

```
$ mytool
bash: mytool: command not found
$ echo $?
127
```

Explain how the shell searched for `mytool`, why it didn't find it, and give two durable ways (and one immediate one) to make the command usable.

Question 13 [Analysis]

The "12-factor" methodology requires configuring an application through **environment variables** rather than through hand-edited files. Relying on what you know about parent → child inheritance and the life cycle of a process, explain why this choice suits disposable, restartable processes perfectly — as your Spring Boot containers will be in lab 08.